

ENGR 421/DASC 521: Introduction to Machine Learning

Homework 02: Discrimination by Regression

Deadline: November 10, 2025, 11:59 PM

In this homework, you will implement a discrimination by regression algorithm for multiclass classification using Python. Here are the steps you need to follow:

1. Your discrimination by regression algorithm will be developed using the following modifications to the linear discrimination algorithm with the softmax function we discussed in the lectures.
 - a. Instead of the softmax function, you are going to use K sigmoid functions to generate $\hat{y}_{ic}$ values. Please note that, in such a case, the summation of $\hat{y}_{ic}$ values is not guaranteed to be 1. However, you are going to pick the largest value to predict the class label.
 - b. Instead of minimizing the negative log-likelihood (i.e., $-\sum_{i=1}^N \sum_{c=1}^K y_{ic} \log(\hat{y}_{ic})$), you are going to use the sum squared errors as the error function to minimize (i.e., $0.5 \sum_{i=1}^N \sum_{c=1}^K (y_{ic} - \hat{y}_{ic})^2$). Please note that you need to find the correct update equations for this modified model.
2. You are given a multivariate classification data set, which contains 310 and 100 bird sound recordings. Each recording is represented as a 20-dimensional real vector after extracting mel-frequency cepstral coefficients (MFCCs). These recordings are from 3 distinct bird types, namely, 1: *Columba palumbus*, 2: *Parus major*, and 3: *Turdus merula*. You are provided with four data files:
 - a. `bird_sounds_features_train.csv`: training recordings,
 - b. `bird_sounds_labels_train.csv`: training labels,
 - c. `bird_sounds_features_test.csv`: test recordings,
 - d. `bird_sounds_labels_test.csv`: test labels.
3. Implement a sigmoid function that calculates K sigmoid outputs for the given data points using the model parameters. (10 points)
4. Implement a one-hot encoding function that converts the given class labels into binary vectors of size K . (10 points)
5. Implement two gradient functions that calculates the partial derivatives of the error function with respect to the model parameters. (20 points)
6. Implement the discrimination by regression algorithm using the implemented sigmoid and gradient functions. (40 points)

```
W, w0, objective_values = discrimination_by_regression(X_train, Y_train,
                                                    W_initial, w0_initial)

print(W)
print(w0)
print(objective_values[0:10])
```

```

[[ 1.59381421 -0.8342634  0.36138484]
 [ 5.69204606 -2.24886419 -0.60119914]
 [ 1.35573616 -1.6589343 -0.2564894 ]
 [ 1.83269495  0.45853047 -0.54873015]
 [-0.42595953 -2.2593673  1.38273361]
 [-2.48430048  2.66826475 -0.57144668]
 [-0.30152494  4.60761372 -1.2191668 ]
 [-1.06275005 -2.92060568  2.64826902]
 [-3.92222633  0.44389396  1.09273121]
 [-0.50058321  2.95475526 -1.04331606]
 [ 2.78319704 -3.60226588  0.68107039]
 [-0.70690224 -1.30636975  0.17183347]
 [-1.65451077  2.7018239 -0.16927799]
 [ 1.57579102 -0.5266817 -0.61602992]
 [-2.02095532 -1.17222946  0.49311231]
 [ 4.49892258 -2.66868855  0.12450483]
 [ 3.1047415 -0.49593492 -2.13500806]
 [ 1.0613569  0.31415337 -0.6548977 ]
 [-2.3550926  0.36262972  0.76173058]
 [-2.15394579  1.4371495  0.34110917]]
[[-0.83182838 -5.1528422 -2.41542809]]
[116.21350229  67.46124331  49.54504071  47.62874391  47.00125196
 46.65118487  44.51729934  43.35423039  42.90680417  42.25245012]

```

7. Calculate the predicted class labels for the data points in training and test sets using the learned model parameters. (10 points)

```

y_hat_train = calculate_predicted_class_labels(X_train, W, w0)
print(y_hat_train[0:10])

```

```

y_hat_test = calculate_predicted_class_labels(X_test, W, w0)
print(y_hat_test[0:10])

```

```
[1 1 3 1 1 1 2 2 1 2]
```

```
[3 1 3 2 1 1 3 3 1 3]
```

8. Calculate the confusion matrices for the data points in training and test sets using the true and predicted class labels. (10 points)

```

[[129  6  6]
 [ 2 68  8]
 [ 5  9 77]]

```

Training accuracy is 88.39%.

```

[[39  1  5]
 [ 4 17  4]
 [ 6  4 20]]

```

Test accuracy is 76.00%.

What to submit: You need to submit your source code in a single file (.py file). You are provided with a template file named as 0099999.py, where 99999 should be replaced with your 5-digit student number. You are allowed to change the template file between the following lines.

```
# your implementation starts below
```

```
# your implementation ends above
```

How to submit: Submit the file you edited to LearnHub by following the exact style mentioned. Submissions that do not follow these guidelines will not be graded.

Late submission policy: Late submissions will not be graded.

Cheating policy: Very similar submissions will not be graded.
